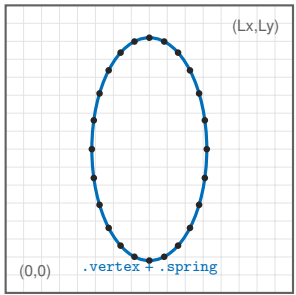
INTRODUCTION TO IB2D AND ITS FILE FORMATS

What IB2d is and when to use it; getting it in MATLAB or Python; an example folder, `input2d` line by line, and every structure file (`.vertex`, `.spring`, `.beam`, `.target`, `.mass` and the rest); where the output goes



Laura A. Miller

Department of Mathematics, University of Arizona

Using IB Software • Part II • Chapter 4a

Claude (Anthropic) was used to organize, combine, and format more than fourteen years (2012–2026) of the instructor's tutorial notes from courses and workshops at several universities.

WHAT YOU WILL BE ABLE TO DO AFTER THIS CHAPTER

- Say what IB2d is, what it models, what it does not, and when to reach for it instead of IBAMR (Part III) or IBFE (Part IV).
- Get the code and run its rubber-band example in MATLAB or in Python, and know what the run prints and writes.
- Read every line of `input2d`: the fluid, time, grid, structure, output and name blocks, and how the file is parsed.
- Write the structure files by hand or from a script: `.vertex` (required) and `.spring`, `.d_spring`, `.beam`, `.nonInv_beam`, `.target`, `.mass`, with the force law behind each, and know the names and columns of the rest.
- Know the one difference that bites first: the MATLAB files count points from 1, the Python files from 0.
- Find the output (`viz_IB2d/`, `hier_IB2d_data/`) and read a file of it before chapter 4b opens them in VisIt.

HOW THESE SLIDES WERE CHECKED

> Every format and every rule on these slides was checked against the readers in the IB2d code (`IBM_Driver.m`, `IBM_Driver.py`, `please_Read_input2d_File`, `please_Initialize_Simulation`) and against runs of the four Python rubber-band examples (October 2026). Where a slide and an older deck disagree, the slide follows the code and says so.

- The force laws are transcribed from `please_Find_Lagrangian_Forces_On_Eulerian_grid.m`, the delta function from `give_Delta_Kernel.m`, the parser's rules from `please_Read_input2d_File` and the initializer, the output names from the driver's `print_vtk_files`.
- The runs: `Rubberband_with_Springs`, `_with_Beams`, `_with_Damped_Springs` and `Wobbly_Beam` from `pyIB2d/Examples`, in WSL (Ubuntu 24.04, Python 3.12, numba 0.68), plotting off; the MATLAB version was read, not run (no MATLAB on the computer used; the Octave attempt is on slide 15).
- The code read is the clone of 2024-03-06; the GitHub page and its news were checked in October 2026 and the repository has moved on since (973 commits), so a key name or an example may differ in detail from these slides. The checks are easy to repeat: the file names on slide 65.

WHERE THIS CHAPTER COMES FROM (1/2)

- "Getting Running on IB2D and IBAMR" (2016, 10 slides): the quick start of the UNC course: clone, "type `main2d`", what IB2d can do, where the output goes. Its cluster part (Kure, LSF, `bmatlab`) is retired (Part X).
- "A quick intro to ib2d" (Lecture 9, 2021–24, 51 slides) and "ib2d and moving the boundaries" (Lecture 11): the boundary description, the four IB steps, the input files, the file formats with their format pictures, the rubber-band generator, homework 4, and (in L9) a VisIt walk-through that belongs to chapter 4b.
- "Introduction to IB2D" (2024, 16 slides): the same file-format slides, the three papers and the four video tutorials.

WHERE THIS CHAPTER COMES FROM (2/2)

- "IB2d and IBAMR associated files" (Lecture 2, 2022, 38 slides): the IB2d/IBAMR comparison (one list vs many files, index base, the domain's origin, $\Delta s = \Delta x/2$), the generator in both flavours, the MATLAB checkers, the project goals.
- "ib2d part 2" (Lecture 10, 2023): MATLAB vs Python, the Anaconda route.
- The IB2d repository (https://github.com/nickabattista/IB2d; the code read here is the 2024-03-06 clone, the page checked October 2026): the README, `IB2d_New_Functionality.txt`, the example folders, and the readers in `IBM_Blackbox`.

What is new here: the Python/MATLAB index difference, the input parser's rules (unknown keys, duplicates, defaults), the force laws as coded, the output file formats, timings on a desktop PC, and the beams-example trap (slide 57).

WHAT IB2D IS

- A two-dimensional immersed boundary code with complete implementations in MATLAB and in Python, written by Nick Battista (The College of New Jersey; started at UNC in May 2015), with the Python port by Christopher Strickland (2016).
- One Cartesian grid, periodic in both directions, solved with FFTs; one list of Lagrangian points carrying springs, beams, target points, masses, muscles and more (slide 11).
- Everything is a text file: `input2d` and a family of `<name>.vertex`, `.spring`, `...`; the output is VTK text that VisIt and ParaView read.
- "Over 75 built-in examples" (the README): rubber bands, channels, heart tubes, jellyfish, insects, corals, cells, a Turek–Hron flag.
- Licence GPL-3.0; 973 commits and 206 stars on GitHub in October 2026.

Where it sits (chapter 1b's table):

IB2d	2D, MATLAB/Python, a laptop, Re up to about 10^3; learning, elastic and muscle-driven boundaries, porous media
IBAMR	2D/3D, C++ on a cluster, adaptive grids, the same fibre models and more (Part III)
IBFE	finite-element solids in IBAMR, 3D organs (Part IV)

The 2024 deck: "useful for 2D simulations of flexible, moving boundaries at Re $<$ 1000."

THE PAPERS AND THE VIDEOS

Cite these when you use IB2d (the README asks for all three, and to let Nick Battista know):

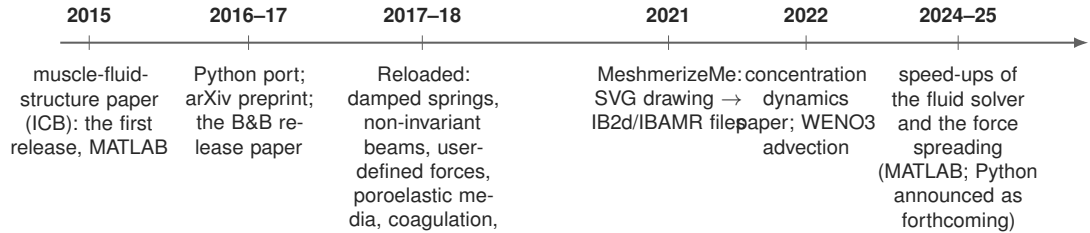
1. N. A. Battista, A. J. Baird, L. A. Miller. A mathematical model and MATLAB code for muscle-fluid-structure simulations. *Integr. Comp. Biol.* 55(5):901–911, 2015 (the muscle models, slide 11; chapter 4e).
2. N. A. Battista, W. C. Strickland, L. A. Miller. IB2d: a Python and MATLAB implementation of the immersed boundary method. *Bioinspir. Biomim.* 12(3):036003, 2017; preprint arXiv:1610.07944.
3. N. A. Battista, W. C. Strickland, A. Barrett, L. A. Miller. IB2d Reloaded: a more powerful Python and MATLAB implementation of the immersed boundary method. *Math. Meth. Appl. Sci.* 41(18):8455–8480, 2018; preprint arXiv:1707.06928.

Later additions: D. M. Senter et al., MeshmerizeMe, *Bioinspir. Biomim.* 16:016008, 2021 (chapter 4f); M. Santiago, N. A. Battista, L. A. Miller, S. Khatri, passive concentration dynamics in IB2d, *Bioinspir. Biomim.* 17:036003, 2022.

Nick Battista's four video tutorials (YouTube): 1 the IB method, fibre models and IB2d (https://youtu.be/PJyQA0vwbgU); 2 what comes with IB2d, the example folders and the input files (https://youtu.be/jSwCKq0v84s); 3 building a geometry and its `.vertex`/`.spring` files from the rubber band (https://youtu.be/I3TLpyEBXfE); 4 VisIt: the points, colour maps and vectors (https://youtu.be/4D4ruXbeCiQ). Tutorials 2 and 3 cover this chapter; 4 is chapter 4b.

The older decks cite paper 3 as "in press"; it appeared in 2018. "Biomemetics" in the decks is "Biomimetics".

HOW IB2D GREW (FROM IB2D_NEW_FUNCTIONALITY.TXT AND THE README NEWS)



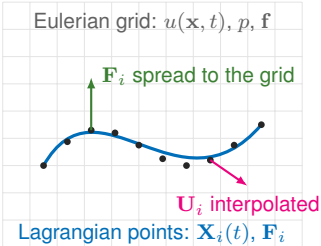
- The input file format changed with Reloaded (the braced blocks of slide 21); the old one-value-per-line format is still accepted (slide 27).
- The MATLAB code has had every feature first; the Python port follows (slide 14).

DESCRIBING THE BOUNDARY

From the 2024 deck, unchanged because it is the whole idea:

- In the immersed boundary method, boundaries are always elastic.
- They can be stiff enough that they act like rigid objects.
- They can be moved in prescribed ways using target points.
- Boundaries have been modelled to resist bending, stretching and twisting.
- All of these properties can change in time, in prescribed ways or according to other models.

In IB2d every one of these is a line in a text file: a spring between two points, a beam over three, a target point tethering one point to a position, a mass attached to one point. The point list itself is the `.vertex` file.



THE FOUR STEPS OF EVERY TIME STEP (AND WHAT IB2D DOES FOR EACH)

1. Forces on the boundary from its deformation: springs, beams, target points, . . . (the laws on slides 37–45); `please_Find_Lagrangian_Forces_On_Eulerian_grid`.
2. Spread the forces to the grid with the discrete delta function (slide 29): $\mathbf{f}(\mathbf{x}) = \sum_i \mathbf{F}_i \, \delta_h(\mathbf{x} - \mathbf{X}_i) \, \Delta s$.
3. Solve the fluid: incompressible Navier–Stokes on the periodic grid by FFT, with Peskin's two-step (half-step) scheme and the advection term in skew-symmetric form; the viscous term is treated implicitly in Fourier space; `please_Update_Fluid_Velocity`.
4. Move the boundary at the local fluid velocity: $\mathbf{U}_i = \sum_{\mathbf{x}} \mathbf{u}(\mathbf{x}) \, \delta_h(\mathbf{x} - \mathbf{X}_i) \, \Delta x \Delta y$, then $\mathbf{X}_i \leftarrow \mathbf{X}_i + \Delta t \, \mathbf{U}_i$; `please_Move_Lagrangian_Point_Positions`.

- Chapter 5 and the theory repository give the equations and the accuracy; chapter 4d does the stability lab (what happens when Δt or the stiffness is too large).
- IB2d's driver (`IBM_Driver`) moves the points a half step with the old velocity, computes the forces once at those positions, solves the fluid once, and moves the points the full step with the new velocity: Peskin's 2002 scheme. The updates of springs, targets and beams (chapter 4c) happen at the start of each step.
- The structure is one list: `xLag`, `yLag` arrays in the order of the `.vertex` file. Every other file refers to that order.

WHAT IB2D CAN MODEL (THE FLAGS IN INPUT2D)

Fibre model	What it is	File, flag
springs (Hookean or power-law)	two points, stiffness, resting length, optional exponent	`.spring`, `springs`
damped springs	the same plus a damping coefficient	`.d_spring`, `damped_springs`
beams (torsional springs)	three points, bending stiffness, a scalar rest curvature	`.beam`, `beams`
non-invariant beams	three points, stiffness, a rest curvature vector (C_x, C_y)	`.nonInv_beam`, `nonInvariant_beams`
target points	a point tethered to a position (prescribed motion in 4c)	`.target`, `target_pts`
mass points, gravity	a massive copy tethered to the point; a gravity vector	`.mass`, `mass_pts`, `gravity`
muscles	force-velocity + length-tension (Hill); the 3-element Hill model	`.muscle`, `FV_LT_muscle`; `.3_muscle`, `3_element_muscle`
porous, poroelastic	a slip velocity through the boundary; a Brinkman-like solid	`.porous`, `porous_media`; `.poroelastic`, `poroelastic`
user-defined force	any force law you code	`.user_force`, `user_force_model`
coagulation	cells that aggregate when close	`.coagulation`, `coagulation`
electrophysiology	FitzHugh–Nagumo activation driving muscles	`electro_phys`

The 3-element Hill model is marked “UNTESTED, likely broken” in the Python README. The gallery and the muscle models are chapter 4e.

WHAT IB2D CAN MODEL (2): THE FLUID-SIDE EXTRAS, AND CHANGING THINGS IN TIME

- Tracers (`tracers`, `.tracer`): passive particles carried by the flow, written with the output.
- A concentration (`concentration`, `.concentration`): a scalar advected and diffused by the flow, with sources; third-order WENO advection since 2022.
- Boussinesq (`boussinesq`, `expansion_coeff`): the concentration acts back on the fluid as buoyancy (`Example_Boussinesq`, Rayleigh–Taylor).
- Brinkman (`brinkman`, `.brinkman`): a permeability field that damps the flow in a region (a porous block without a boundary).
- An imposed body force (`arb_ext_force`): the way the channel examples drive flow ("pseudo-inflow", the README), since the domain has no inflow boundary (slide 28).
- Non-square grids (`Lx` $\neq$ `Ly`, `Nx` $\neq$ `Ny`) since Reloaded; restart after a crash (MATLAB; `Example_Test_Restart_Protocol`).
- Each fibre model can also be updated in time (`update_springs`, `update_target`, `update_beams`, `update_nonInv_beams`, `update_damp_springs`) by a function you write: chapter 4c.

GETTING THE CODE

```
git clone https://github.com/nickabattista/IB2d.
    git
# or: Code > Download ZIP on the GitHub page
```

- The repository is big because the MATLAB examples ship their data: about 1.5 GB on disk for `matIB2d` after a clone, plus 0.2 GB of history; the ZIP is about 250 MB (a November 2024 copy).
- No build, no install: MATLAB runs the `.m` files; Python needs the packages of slide 16.
- Keep your own examples outside the clone (copy an example folder and rename it), so a `git pull` never touches them.

```
IB2d/
  README.md                  # overview, news, the videos
  IB2d_New_Functionality.txt # what came after the 2017 paper
  IB2d_To_Do_List.txt
  license                    # GPL-3.0
  matIB2d/
    IBM_Blackbox/            # the solver (do not edit)
    Examples/                # 30 folders, 75+ examples
    Testing/
  pyIB2d/
    IBM_Blackbox/            # the Python solver
    Examples/                # 16 examples
    pyIB2d.py                # a front end (-p path, -d dialog)
    README.txt
  data_analysis/             # MATLAB and Python readers of the
      output
    analysis_in_matlab/  analysis_in_python/
```

MATLAB OR PYTHON?

	matIB2d	**pyIB2d**
examples	30 folders, over 75 examples, including the teaching sets `Examples_Education` (convergence, interpolation, pendulum) and `Examples_First_Year_Seminar` (corals, jellyfish, leaf, nematocyst, swimmer, . . .)	16: the rubber bands (springs, beams, damped springs), wobbly beams, heart tube with muscle, tracers in an impedance pump, porous and poroelastic bands, concentration diffusion, Rayleigh–Taylor, thrips, Turek–Hron, a user-defined fibre model
features	everything on slides 11–12; restart after a crash; the 2024–25 speed-ups	the core models; concentration, poroelastic and Brinkman flags exist; Lecture 10: “there may also be some functionality that doesn’t exist yet in Python (e.g. chemical concentrations)”
speed and plotting	R2020a/b break the live plotting (use VisIt instead; README)	needs numba (“the code crawls without it: 600 % speed-up”); optional `pyfftw` (≈20 %) and `vtk` or the C writer for faster output
indices	points count from 1 in every structure file	points count from 0 in every structure file (slide 35)

Lecture 10’s advice stands: “You may use either version. There are more examples in MATLAB. Converting examples from MATLAB to Python isn’t too terrible.” This chapter shows both; the runs and pictures are from the Python version on a desktop PC (WSL, Python 3.12, numba 0.68).

RUNNING EACH VERSION; AND OCTAVE

- MATLAB: open the example folder in MATLAB and type `main2d`. A figure window opens when `plot_Matlab = 1`.
- Python: `python main2d.py` in the example folder; or, from anywhere, `python pyIB2d.py -p <path to input2d>` (`-d` opens a file dialog instead). The front end writes the output next to `pyIB2d.py`, not in the example folder (its README says so).
- Octave is not supported, and the attempt is worth a line: the MATLAB driver asks `ver('MATLAB')` for the release year (to choose between `exist` and `isfile`) and stops under Octave 8.4 with "invalid number of elements on RHS of assignment". With that check patched out, the rubber band stopped at the first step with "BLOW UP! (*forces TOO large*)": Octave's `textscan` fills the five-column spring format from a four-column file differently, so the stiffness column held garbage. Use MATLAB or the Python port.
- Either version runs on a cluster node in batch (chapter 1): MATLAB with `matlab -batch main2d` in a job script, Python with `python3 main2d.py`; the output folders appear in the working directory.

SETTING UP THE PYTHON SIDE

Anaconda (Lecture 10's route, any OS): install Anaconda, then in the Anaconda prompt or a terminal:

```
conda install numpy matplotlib numba
pip install pyfftw      # optional, faster FFTs
conda install -c conda-forge vtk   # optional,
    faster output
```

A plain Python 3 (what the runs here used: Ubuntu 24.04 in WSL, Python 3.12):

```
sudo apt install python3-numpy python3-matplotlib
     python3-vtk9 python3-pip
python3 -m pip install numba        # 0.68 in
    October 2026
python3 -c "import numba; print(numba.__version__
    )"
```

- `numba` compiles the delta-function loops (`@jit(nopython=True)` in `Supp.py`); without it a run takes many minutes instead of seconds.
- The start-up banner says what it found: "Running without pyFFTW library" and a note about `vtk` are informational, not errors.
- On Windows, Anaconda is the easiest path; on a cluster, load a Python module and `pip install --user numba` (chapter 1).
- `plot_Matlab = 1` in `input2d` opens a matplotlib window during the run; set it to 0 on a headless machine (or set `MPLBACKEND=Agg`).

AN EXAMPLE FOLDER: RUBBERBAND_WITH_SPRINGS

```
matIB2d/Examples/Example_Standard_Rubberband/
                  Rubberband_with_Springs/
  main2d.m            # run this
  input2d             # all parameters
  Rubberband.m        # writes the two files
     below
  rubberband.vertex   # 64 points
  rubberband.spring   # 64 springs
pyIB2d/Examples/Rubberband_with_Springs/
  main2d.py  input2d  Rubberband.py
  rubberband.vertex  rubberband.spring
```

After a run, two folders appear next to them:

```
viz_IB2d/          # fields and points, one
     set per dump
hier_IB2d_data/    # forces on the points, one
      set per dump
```

- `main2d` reads `input2d`, adds `IBM_Blackbox` to the path (MATLAB tries `../IBM_Blackbox` up to four levels up; Python appends `../../IBM_Blackbox`) and calls `IBM_Driver`. Move an example two levels deeper and the MATLAB path still works; the Python one needs its `sys.path.append` line changed.
- `input2d`: slides 21–26.
- `Rubberband.m` / `.py` (other examples name it after the geometry: `BeamCurve.m`, `Make_Jelly_Geometry.m`, `make2d.py`; the course decks call it `generate_mesh2d.m`) builds the geometry and writes the `.vertex` and `.spring` files; it is not called by the run. Change it, run it, then run `main2d` (slide 50).
- The structure files are read from the folder you run in; their base name is the `string_name` of `input2d`.

RUNNING IT

MATLAB: open the folder and type `main2d`. Python:

```
cd pyIB2d/Examples/Rubberband_with_Springs
python3 main2d.py
```

What the four Python rubber-band examples took on a desktop PC under WSL (numba, output on, plotting off; numpy free to use several threads, which the 64^2 run did):

example	grid	Δt, T	steps	points	wall time
Rubberband_with_Springs	32^2	10^{-3}, 1.5	1500	64	3.9 s
Rubberband_with_Damped_Springs	32^2	10^{-3}, 1.5	1500	64	3.3 s
Wobbly_Beam	32^2	$5 \cdot 10^{-5}$, 0.05	1000	64	2.7 s
Rubberband_with_Beams	64^2	10^{-2}, 25	2500	249	12.7 s

WHAT A RUN COSTS, AND WHAT TO READ FIRST

- Seconds, not hours: the whole point of IB2d for learning. The cost grows with $N_x N_y \log(N_x N_y)$ for the FFT and with the number of points for the spreading; a 512^2 run of the same length is minutes to an hour.
- The timings are wall-clock with numba and the output on (the 64^2 run used several threads: its CPU time was five times the wall time); the live plot (`plot_Matlab = 1`) adds a window redraw at every dump and can double the wall time on a small grid.
- Read the “FIBER MODEL INCLUDES” lines every time (next slide). They list the models the code actually switched on; if a model you meant is missing, the flag was not read (slide 57).

WHAT THE RUN PRINTS

```
Running without pyFFTW library.
---------------->>      IB2d      <<----------------
If using the code for research purposes please cite the following two papers: [1] ... [2] ... [3] ...
 |****** Prepping Immersed Boundary Simulation ******|
--> Reading input data for simulation...
[ 1.  0.  0. ... -1. ... 1.  0.  0.  0.]       # the structure flags as read, in the order of the keys
--> FIBER MODEL INCLUDES:
  - Springs and ...
                  NOT dynamically updating spring properties
--> Background Flow Items
      (No tracers nor other passive scalars immersed in fluid)
 |****** Begin IMMERSED BOUNDARY SIMULATION! ******|
Current Time(s): 0.0
Current Time(s): 0.020000
...
Current Time(s): 1.500000
```

The Python console, abridged; MATLAB prints the same lines without the banner's package notes and the flag vector, adds a CFL line after each time, and the example's `main2d.m` ends with "IMMERSED BOUNDARY SIMULATION HAS FINISHED!". A time is printed at every dump, so a run that is silent for long is either large or stuck.

INPUT2D, BLOCK BY BLOCK: FLUID, TIME, GRID

```
%*****% INPUT2d FILE: %*****%
% FLUID PARAMETER VALUES %
Fluid_Parameters {
mu = 0.01               % Dynamic Viscosity (N*s/m^2)
rho = 1                 % Density (kg/m^3)
}
% TEMPORAL INFORMATION %
Temporal_Information {
Tfinal = 1.5            % Final time of simulation (s)
dt = 1.0e-3             % Time-step (s)
}
% GRID PARAMETERS %
Grid_Parameters {
Nx = 32     % # of Eulerian Grid Pts. in x-Direction (MUST BE
      EVEN!!!)
Ny = 32     % # of Eulerian Grid Pts. in y-Direction (MUST BE
      EVEN!!!)
Lx = 1.0    % Length of Eulerian Grid in x-Direction (m)
Ly = 1.0    % Length of Eulerian Grid in y-Direction (m)
supp = 4    % Choose dirac-delta support (KEEP IT EVEN!!!)
}
```

- `mu`, `rho`: dynamic viscosity and density. The units in the comments are nominal; the examples are effectively dimensionless (slide 60).
- `Tfinal`, `dt`: the run is `while current_time < Tfinal`, so $T/\Delta t$ steps. Stability sets Δt (chapter 4d); the examples use 10^{-3} at 32^2 with $k = 2.5 \cdot 10^4$, and $5 \cdot 10^{-5}$ for the stiff beam.
- The grid block: next slide.

INPUT2D: THE GRID BLOCK

```
Grid_Parameters {
Nx = 32      % (MUST BE EVEN!!!)
Ny = 32      % (MUST BE EVEN!!!)
Lx = 1.0     % Length in x (m)
Ly = 1.0     % Length in y (m)
supp = 4     % dirac-delta support (KEEP IT
     EVEN!!!)
}
```

$\Delta x = L_x/N_x$, $\Delta y = L_y/N_y$; here $1/32$.

- `Nx`, `Ny`, `Lx`, `Ly`: the grid; even counts for the FFT; `Lx` $\neq$ `Ly` and `Nx` $\neq$ `Ny` are allowed since Reloaded.
- `supp = 4`: the width of the discrete delta function in grid cells (slide 29); leave it.
- The grid is periodic in x and in y, and the box is $[0, L_x] \times [0, L_y]$ with the origin at the lower-left corner (slide 28).
- Resolution is the main cost knob: 64^2 is four times the work of 32^2 per step, and the stable Δt shrinks with Δx (chapter 4d), so eight times in all.

INPUT2D: THE STRUCTURE FLAGS

```
% LAGRANGIAN STRUCTURE MODEL INFORMATION %
Lag_Structure_Info {
springs = 1              % 0 (for no) or 1 (for
     yes)
update_springs = 0
target_pts = 0
update_target = 0
beams = 0
update_beams = 0
nonInvariant_beams = 0   % "non-Invariant Beams"
update_nonInv_beams = 0
FV_LT_muscle = 0         % Force-Velocity & Length
     -Tension Model
3_element_muscle = 0     % Hill 3-Element Model w/
      FV-LT Model
arb_ext_force = 0
tracers = 0
mass_pts = 0
gravity = 0              % PLEASE CHECK GRAVITY
     VECTOR BELOW
x_gravity_vec_comp = 0   % x-Component of
     Gravity Vector
y_gravity_vec_comp = -1 % y-Component of
     Gravity Vector
porous_media = 0
poroelastic = 0
electro_phys = 0
damped_springs = 0
update_damp_springs = 0
boussinesq = 0           % FOR BOUSSINESQ
     APPROXIMATION!
expansion_coeff = 1.0    % expansion coefficient
      for Boussinesq approx.
user_force_model = 0
}
```

The block of the springs example, as shipped (comments shortened). Every flag is 0 or 1 except the two gravity components and the expansion coefficient.

THE STRUCTURE FLAGS: WHAT THEY DO

- Each `= 1` makes the code read the matching file (`<name>.spring`, ...; slide 31) and apply that force. A file that is present but whose flag is 0 is ignored.
- The `update_*` flags make the driver call your `update_Springs`, `update_Target_Point_Positions`, `update_Beams`, `update_nonInv_Beams` or `update_Damped_Springs` function every step (chapter 4c).
- `gravity = 1` needs `mass_pts = 1` (gravity acts on the massive copies) and the two gravity components, which the driver normalises to a direction; the magnitude $g = 9.80665$ is hard-coded in `please_Update_Massive_Boundary_Velocity`, so the examples' units are SI there whatever the rest of the file does.
- Both versions also know `brinkman` here and MATLAB `coagulation`; the Python examples add `concentration = 0/1` to this block, while the MATLAB examples that use it have their own block (slide 26).
- A missing flag is 0. The only other keys: the output and name blocks.

INPUT2D: OUTPUT AND NAME

```
% PRINTING INFORMATION
Output_Info {
print_dump = 20      % How often to print (PLOT)
     to screen
save_Vorticity = 1   % Save Vorticity Scalar Data
     to .vtk format
save_Pressure = 1
save_uVec = 1        % the velocity vector field
save_uMag = 1
save_uX = 0
save_uY = 0
save_fMag = 0        % force magnitude on the grid
save_fX = 0
save_fY = 0
save_hier = 1        % force data on the structure
      (hier_IB2d_data)
```

```
plot_Matlab = 1      % Plot in Matlab? (1=YES,
     0=NO)
plot_LagPts = 1
plot_Velocity = 1
plot_Vorticity = 1
plot_MagVelocity = 0
plot_Pressure = 0
}
% LAGRANGIAN STRUCTURE NAME %
Lag_Name {
string_name = "rubberband"   % String name for
     simulation
}
```

The last two blocks of the same file.

OUTPUT AND NAME: WHAT THE KEYS MEAN

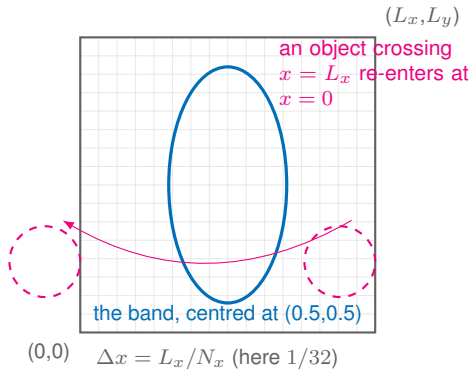
- `print_dump`: write (and plot) every this many steps; 1500 steps at 20 gave 76 dumps (`0000` at $t = 0$, then every 0.02 s). The MATLAB examples sometimes spell it `print_Dump`; both are accepted.
- `save_*`: which fields go to `viz_IB2d/` (slide 53). A `save_*` key that is missing counts as 1 (on); a `plot_*` key that is missing counts as 0.
- `plot_*`: the live figure (MATLAB figure or matplotlib). Off for batch runs.
- `string_name`: the base name of every structure file. `"rubberband"` means `rubberband.vertex`, `rubberband.spring`, ... in the run folder. The quotes are required.
- MATLAB's concentration examples add a block `Concentration_Info { concentration = 1, kDiff = [...], advection = 0/1, source = 0/1, num_con = 2 }` and `plot_Concentration` (`Example_Concentration_Dynamics`).

HOW INPUT2D IS READ (BOTH VERSIONS)

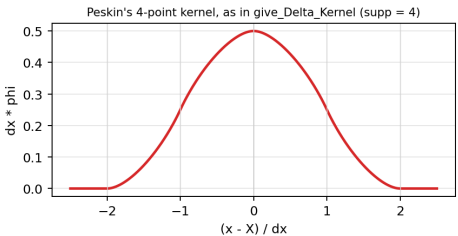
- A temporary copy is made with everything after a % or a # removed, line by line (so both comment characters work, and a comment may follow a value).
- A line with `{` opens a block; inside it every `key = value` is stored under the block's name until the `}`. A value in double quotes is a string; anything else goes through `str2num`/`float` (so `[8.7e-4 8.7e-4]` is a vector in MATLAB).
- The initializer then looks keys up by name inside each block. The header's warnings "DO NOT SWAP ROWS" and "KEEP SPACES BETWEEN '='s" date from the old fixed-order parser; with the braced format the order inside a block does not matter.
- If the file has no `{` at all, MATLAB reads the old format (one `key = value` per line, no blocks); the Python port's fallback is commented out and it stops with `ValueError: 'Fluid_Parameters' is not in list`.
- A key the code does not know is ignored, silently: no warning, no error (slide 57).
- A key that appears twice in a block: the Python reader took the first value in a test; in MATLAB the lookup returns two entries and the initializer's `try`/`catch` stops with "LAGRANGIAN STRUCTURE INFO Improperly Declared in input2d file" (from the code, not run).
- A block that is missing stops the run with a raw error (an empty-cell index in MATLAB, a `ValueError` traceback in Python); a required key missing inside a present block (`mu`, `rho`, `Nx` ..., `string_name`) gives the readable "FLUID Parameters Improperly Declared in input2d file" and its siblings; an optional key missing takes its default (structure flags 0, `save_*` 1, `plot_*` 0).

THE DOMAIN: PERIODIC, ORIGIN AT THE CORNER

- The box is $[0, L_x] \times [0, L_y]$. The bottom-left corner is the origin (Lecture 2); the structure must fit inside, with room for the flow it makes. (IBAMR's box can be anywhere; its examples put the origin at the centre: chapter 7.)
- The FFT solver makes both directions periodic: what leaves through the right edge enters from the left; the driver wraps the points (`xLag = mod(xLag, Lx)`). There are no wall boundary conditions.
- Walls are therefore built from target points (the channel examples: two rows of tethered points), and inflow is an imposed body force (`arb_ext_force`, the "pseudo-inflow" of the README) rather than a boundary condition.
- A structure that reaches an edge interacts with its own periodic image. The Python README: "this code may BLOW UP when Lagrangian points cross a boundary."



THE DELTA FUNCTION



`give_Delta_Kernel`: Peskin's 4-point function,

$$\phi(r) = \tfrac{1}{8}\big(3 - 2|r| + \sqrt{1 + 4|r| - 4r^2}\big), \qquad |r| < 1,$$
$$\phi(r) = \tfrac{1}{8}\big(5 - 2|r| - \sqrt{-7 + 12|r| - 4r^2}\big), \quad 1 \le |r| < 2,$$

zero beyond; $\delta_h(\mathbf{x}) = \phi(x/\Delta x)\,\phi(y/\Delta y)/(\Delta x \Delta y)$. `supp = 4` is its width in cells.

- The force of a point is spread over the 4×4 cells around it, and its velocity is averaged from them: the boundary is a layer about two cells thick.
- The weights of the four cells sum to one wherever the point sits (checked for the kernel above).

HOW FAR APART THE POINTS SHOULD BE

- Points farther apart than about Δx leave gaps the fluid leaks through; much closer than $\Delta x/2$ buys nothing. The rule of Lecture 2: $\Delta s \approx \Delta x/2$.
- The rubber-band generator takes $N = 2N_x$ points, which is right on average, but spaces them evenly in angle on an ellipse: Δs runs from $0.63\,\Delta x$ at the ends of the long axis to $1.25\,\Delta x$ at the sides (slide 34). Space points by arc length when you write your own (chapter 4c), and print Δs from the `.vertex` file (the differences of consecutive points, three lines in either language) next to Δx.
- Chapter 5 says why (the discrete delta function and its moment conditions); chapter 4d shows what the spacing does to the results.
- The same rule holds for IBAMR's fibre structures ($\Delta s = \Delta x/2$) and differs for IBFE meshes ($\Delta s \approx \Delta x$): Lecture 2's one-line summary, chapter 18 for the finite-element side.

THE STRUCTURE FILES: ONE NAME, MANY EXTENSIONS (1/2)

File	Flag	Columns after the count line	Slide
`.vertex`	always read	`x y` per point	33
`.spring`	`springs`	`i j k L [alpha]`	37
`.d_spring`	`damped_springs`	`i j k L b`	39
`.beam`	`beams`	`i j k k_b C`	40
`.nonInv_beam`	`nonInvariant_beams`	`i j k k_b Cx Cy`	42
`.target`	`target_pts`	`i k_t`	43
`.mass`	`mass_pts`	`i k_m m`	45
`.muscle`	`FV_LT_muscle`	`i j LFO SK a b Fmax`	46
`.3_muscle`	`3_element_muscle`	`i j LFO SK a b Fmax kSpr alpha` (Python reads `.muscle`)	46

Every file in this table starts with one line holding the number of entries, then one entry per line, space-separated, written with `%1.16e`. The counts are read but the readers take every line that follows, so the count must match. `i`, `j`, `k` are point indices in the order of the `.vertex` file.

THE STRUCTURE FILES: ONE NAME, MANY EXTENSIONS (2/2)

File	Flag	Layout	Slide
`.porous`	`porous_media`	count, then `i alpha flag`	46
`.poroelastic`	`poroelastic`	count, then `i alpha`	46
`.tracer`	`tracers`	count, then `x y` per tracer	46
`.concentration`	`concentration`	an $N_y \times N_x$ array of values, no count line	47
`.brinkman`	`brinkman`	the same layout	47
`.user_force`	`user_force_model`	one header line, then whatever your force function reads	47
`.coagulation`	`coagulation`	one value per line: start index, radius, force threshold, cell sizes	47
`.geo_connect`	read if present	`i j` pairs, no count line: geometric connections for the force and source routines	47

A flag set to 1 without its file stops the run (the reader cannot open the file); a file without its flag is ignored. All of them are read once, at the start; the `update_*` functions of chapter 4c change the arrays in memory, not the files.

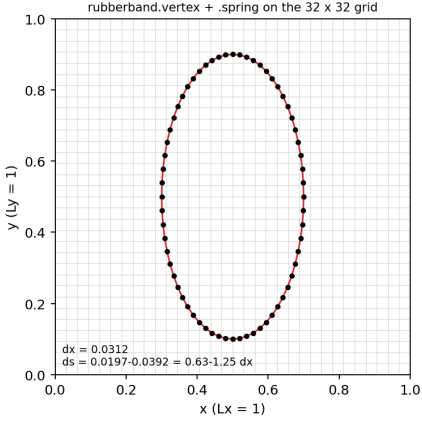
.vertex: the points

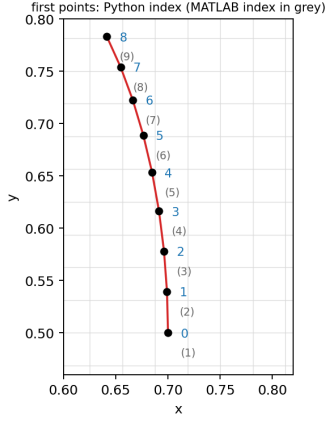
```
64
6.9999999999999996e-01 5.0000000000000000e-01
6.9903694533443939e-01 5.3920685613182429e-01
6.9615705608064604e-01 5.7803612880645128e-01
...
```

The shipped `rubberband.vertex`: 64 points on an ellipse centred at (0.5, 0.5) with semi-axes 0.2 (x) and 0.4 (y), starting at the rightmost point and going counter-clockwise.

- Line 1: the number of points N. Then N lines of `x y`, in metres or whatever unit the grid uses.
- The order is the point index used by every other file: the first point is 0 in Python, 1 in MATLAB (slide 35).
- Several boundaries go into one list: two plates of 100 points are points 1–100 and 101–200 (MATLAB), and you keep track of where one ends (Lecture 2). IBAMR takes one `.vertex` per structure instead (chapter 6).
- The reader takes `%f %f` pairs: no comments, no blank lines, no third column.

THE EXAMPLE'S POINTS ON THE GRID





The 64 points on the 32×32 grid (left; the red segments are the `.spring` entries) and the first nine with both numberings (right: Python's in blue, MATLAB's in grey). The spacing is uneven because the generator spaces the points in angle, not in arc length (slide 30).

INDICES: MATLAB COUNTS FROM 1, PYTHON FROM 0

The same example, the shipped `rubberband.spring`:

```
matIB2d (1-based)          pyIB2d (0-based)
64                         64
1 2 2.5e+04 0.0e+00        0 1 2.5e+04 0.0e+00
2 3 2.5e+04 0.0e+00        1 2 2.5e+04 0.0e+00
...                        ...
63 64 2.5e+04 0.0e+00      62 63 2.5e+04 0.0e+00
64 1 2.5e+04 0.0e+00       63 0 2.5e+04 0.0e+00
```

(values abbreviated). The same holds for `.beam`, `.d_spring` and `.target`: `Wobbly_Beam` tethers points `0` and `63` in Python; the MATLAB generators write `s` for `s = 1:N`.

- Both readers use the numbers as they are: MATLAB indexes `xLag(i)`, Python `xLag[i]`. Neither shifts.
- So a structure file written for one version is off by one in the other: in MATLAB an index 0 is an error (“Index exceeds . . . ” or “Array indices must be positive integers”); in Python an index N is an `IndexError`, and an index 1 where 0 was meant silently connects the wrong points.

INDICES, CONTINUED: PORTING BETWEEN THE VERSIONS

- IBAMR counts from 0, like Python (chapter 6, which has the same warning the other way round). MeshmerizeMe writes only `.vertex` files, which carry no indices (chapter 4f), so the base matters in the files you add to them.
- When you port an example, convert the files, not the reader: `i - 1` on every index column (or `+ 1` the other way), in a script, and re-run the geometry checker (slide 52).
- The count line and the value columns do not change; only the index columns do: 2 of them in `.spring` and `.d_spring`, 3 in the beam files, 1 in `.target`, `.mass`, `.porous`, `.poroelastic`, 2 in `.muscle` and `.geo_connect`.
- The output does not care: the VTK files number points from 0 in both versions (VTK's convention), which is one more reason to check the structure in the output's first dump rather than in the input files.

> The 2024 "Introduction to IB2D" deck shows the MATLAB numbering ("vertex 1 . . . N"); Lecture 9 and Lecture 2 (2022) show `x_0 ...x_{N-1}`. Both are right, for their version.

`.spring`: two points, a stiffness, a resting length, an exponent

```
64                          # number of springs
1 2 2.5e+04 0.0e+00         # i j k L  (MATLAB
    indices)
2 3 2.5e+04 0.0e+00
...
64 1 2.5e+04 0.0e+00        # closes the band
```

The force on point i from the spring to j, with $\mathbf{d} = \mathbf{X}_j - \mathbf{X}_i$:

$$\mathbf{F}_i = \tfrac{1}{2}(\alpha+1)\, k \left(|\mathbf{d}| - L\right)^{\alpha} \frac{\mathbf{d}}{|\mathbf{d}|},$$

and $\mathbf{F}_j = -\mathbf{F}_i$; so $\alpha = 1$ is Hooke's law $k(|\mathbf{d}| - L)$ (`give_Me_Spring_Lagrangian_Force_Densities`).

- Columns: `i j k L` and an optional fifth, `alpha`, the degree of nonlinearity. Missing: 1 (linear). MATLAB's `textscan` returns `NaN` for a missing fifth value and the reader fills 1 (a four-column file works as shipped); the Python reader accepts `NaN`/`NA`/`NULL` markers and fills 1, and stops with a message if rows have different lengths.
- `L = 0` (this example) makes a band that wants to shrink to a point; the fluid inside resists, and the band settles near a circle of the enclosed area. `L = ds` (the generator of Lecture 9) makes a band at rest in its initial length.

SPRINGS, CONTINUED

- Any network is allowed: no limit on springs per point, diagonals, cross-links. A spring over the periodic edge is handled inside the force routine (a separation longer than half the box is shortened by L_x or L_y); only the beams have a named check function.
- Units: `k` is a force per length of stretch (N/m), per Lagrangian point; the spreading multiplies by Δs. The convention $\kappa\,\Delta s/\Delta s^2$ of Lecture 9's generator makes the stiffness independent of the resolution; chapters 6 and 8b discuss the scaling.
- `NL_Rubberband_With_Springs` is the same band with `alpha = 2` and $k = 10^5$: the force grows with the square of the stretch, a rubber that stiffens.
- With `update_springs = 1`, `update_Springs(dt, current_time, xLag, yLag, springs_info)` is called every step and returns the spring table, so k and L can change in time: the heart-tube examples pump by changing resting lengths (chapters 4c and 4e).

.d_spring: damped springs

```
64
1 2 2.5e+04 0.0e+00 5.0e+00    # i j k L b
2 3 2.5e+04 0.0e+00 5.0e+00
...
```

$$\mathbf{F}_i = k\,(|\mathbf{d}| - L)\,\frac{\mathbf{d}}{|\mathbf{d}|} - b\,\mathbf{U}_i, \qquad \mathbf{F}_j = -\mathbf{F}_i,$$

with $\mathbf{U}_i$ the leader point's own velocity, taken as its displacement since the previous step over Δt (the driver keeps `xLag_P`, `yLag_P` for this); linear only, and the damping is of the first point's motion, not of the relative motion of the pair (both versions).

- Flag `damped_springs = 1`; the file `<name>.d_spring`; five columns, all required.
- `Rubberband_with_Damped_Springs` is the springs example with $b = 5$: the band relaxes the same way (slide 59); at $b = 5$ the two curves are indistinguishable.
- A damped spring is the simplest way to stop a structure ringing when the stiffness has to be high; `update_damp_springs` can change k, L and b in time (chapter 4c).
- Added in Reloaded (2017); not in IBAMR's file set (IBAMR's target points have the damping parameter instead: chapter 6).

.BEAM: TORSIONAL SPRINGS OVER THREE POINTS

```
62                                      #
    Wobbly_Beam (Python)
0 1 2 7.5e+09 0.0e+00                   # i j k  k_b
      C
1 2 3 7.5e+09 0.0e+00
...
249                                     #
    Rubberband_with_Beams
248 0 1 1.0e+07 -1.2851754376167265e-06
0 1 2 1.0e+07 -1.2851754376155979e-06
```

With $\mathbf{p}, \mathbf{q}, \mathbf{r}$ the three points (left, middle, right):

$$c = (x_r - x_q)(y_q - y_p) - (y_r - y_q)(x_q - x_p),$$

the cross product of the two segments, a signed area; C is its rest value. The three forces are $k_b\,(c - C)$ times the perpendiculars of the segments, with the signs as coded:

$$\mathbf{F}_p = k_b(c - C)\,\big(y_r - y_q,\ -(x_r - x_q)\big),$$
$$\mathbf{F}_r = k_b(c - C)\,\big(y_q - y_p,\ -(x_q - x_p)\big),$$
$$\mathbf{F}_q = k_b(c - C)\,\big(y_r - y_p,\ -(x_r - x_p)\big) = \mathbf{F}_p + \mathbf{F}_r$$

(`give_Me_Beam_Lagrangian_Force_Densities`; the three do not sum to zero, unlike the gradient of a bending energy, which would give the end forces the opposite sign).

TORSIONAL SPRINGS, CONTINUED

- Columns: `i j k` (left, middle, right), `k_b`, `C`. The force acts on all three; the code's comment "force is exerted on the middle point" names where it is applied in the bookkeeping.
- `C = 0` is a straight beam at rest (`Wobbly_Beam`: a beam whose ends are target points, bending back after an initial curve). The rubber band's file gives one constant $C = -1.285 \cdot 10^{-6}$ to all 249 beams (its generator computes the curvatures of a second, rounder geometry), while the cross products of its initial ellipse range from $-4.8 \cdot 10^{-6}$ to $-3 \cdot 10^{-8}$: the band is not at rest and relaxes towards a rounded shape. The 2024 deck's "preferred curvature" is this number, in units of length^2, not $1/\text{length}$.
- The model is invariant under rotation (a rotated beam feels no force), unlike the non-invariant beam of the next slide. It is weak against pure stretching, so beams are usually combined with springs.
- A beam cannot start or end at the ends of an open structure: $N - 2$ beams for N points in a line, N for a closed loop. The generator writes one line per interior point.
- IBAMR's `.beam` is the non-invariant kind with a curvature vector (chapter 6, chapter 10).

.NONINV_BEAM: THE BEAM EQUATION WITH A REST CURVATURE VECTOR

```
N_beams
i j k  k_b  Cx  Cy        # six columns
...
```

$$\mathbf{F}_q = 2k_b\,(\mathbf{X}_r - 2\mathbf{X}_q + \mathbf{X}_p - \mathbf{C}),$$
$$\mathbf{F}_p = \mathbf{F}_r = -\tfrac{1}{2}\mathbf{F}_q,$$

the discrete fourth derivative of the 2024 deck's $\mathbf{F} = k_{NIB}\,\partial^4(\mathbf{X} - \mathbf{X}_b)/\partial s^4$ assembled from second differences (`give_Me_nonInv_Beam_Lagrangian_Force_Densities`).

- Flag `nonInvariant_beams = 1`, file `<name>.nonInv_beam`, six columns: the rest "curvature" is the vector $\mathbf{C} = (C_x, C_y)$ of second differences of the preferred shape.
- Non-invariant: the force depends on the orientation of the structure, so a rotated beam is bent back towards its original orientation. Right for a structure clamped in space (a flag, a leaf on a stem: `Wobbly_NonInv_Beam`); wrong for a free swimmer, which needs the torsional beam.
- Same convention as IBAMR's beams and its `Cx Cy` columns (chapter 10), which is why a non-invariant beam file converts to IBAMR by shifting the indices.
- Added in Reloaded (2017), with the explanation quoted in the 2021–24 decks.

.target: tethered points

```
2                       # Wobbly_Beam: the two
     ends
0 2.0e+08               # i  k_t
63 2.0e+08
6552                    # the sawtooth channel
     (chapter 4c)
1 1.0e+11
2 1.0e+11
...
```

$$\mathbf{F}_i = k_t \left(\mathbf{X}_i^{\text{target}} - \mathbf{X}_i \right),$$

a zero-rest-length spring to a position that is, by default, the point's initial position (`give_Me_Target_Lagrangian_Force_Densities`).

- Two columns: the point index and its stiffness. Any subset of the points; a point can be a target point and a spring end at once.
- Target points are how IB2d holds a structure in place (the beam's clamped ends, channel walls, a cylinder) and how it prescribes motion: with `update_target = 1` your `update_Target_Point_Positions` moves the target positions every step and the points follow (chapter 4c's sawtooth: 10^{11} for 6552 wall points).

TARGET POINTS, CONTINUED

- The stiffness is a penalty: too small and the points lag behind their targets (a leaky wall), too large and Δt must shrink (chapter 4d). Start from the examples' values and the grid they use.
- The target positions are kept in the driver's `target_info` array (columns: index, x, y, k_t), which is what the update function edits (Lecture 11): a target point's position is a number in memory, not a point of the `.vertex` file, so moving it costs nothing.
- A rigid object in IB2d is a closed curve of target points; a wall is a line of them; a hinge is a target point at one end of a beam. IBAMR's `.target` adds a damping coefficient (chapter 6) and chapter 9a has the same lesson for IBAMR.

.mass: massive points and gravity

```
N_mass
i  k_m  m           # index, mass-spring
     stiffness, mass
...
```

Each massive point is a separate particle of mass m at $\mathbf{Y}_i$, tied to the boundary point by a stiff spring:

$$\mathbf{F}_i = k_m\,(\mathbf{Y}_i - \mathbf{X}_i),$$
$$m\,\ddot{\mathbf{Y}}_i = -\mathbf{F}_i + m\,\mathbf{g},$$

integrated by the driver (`please_Move_Massive_Boundary`, `please_Update_Massive_Boundary_Velocity`); the boundary point itself still moves with the fluid.

- Flags `mass_pts = 1`, and `gravity = 1` with `x_gravity_vec_comp`, `y_gravity_vec_comp` giving the direction (the examples use $(0, -1)$); the magnitude $g = 9.80665$ is hard-coded in the velocity update of the massive points.
- This is the penalty-IB idea (chapter 13 for IBAMR): the IB point is massless, the inertia sits in its shadow particle. The spring stiffness k_m must be high enough that the two stay together, which again limits Δt.
- Uses: sinking cells, “massive cells racing” under gravity vs. flow (`Example_Mass_Points`, `Gravity_Cellular_Race`), flexible structures with inertia.
- IBAMR’s `.mass` has the same three numbers in a different order (`i mass kappa`): chapter 6.

THE OTHER FILES, BRIEFLY (1/2)

- `.muscle` (`FV_LT_muscle`): `i j LFO SK a b Fmax`: a muscle between two points, with the length of maximal tension `LFO`, a length-tension width `SK`, the Hill force-velocity constants `a`, `b`, and the peak force `Fmax`; activation comes from a function you write (chapter 4e; the 2015 paper).
- `.3_muscle` (`3_element_muscle`): the seven above plus a parallel nonlinear spring `kSpr`, `alpha`. MATLAB reads `<name>.3_muscle`; the Python port reads `<name>.muscle` for it and marks it untested.
- `.porous` (`porous_media`): `i alpha flag`: a porosity coefficient per point and a stencil flag; the points slip through the fluid in proportion to the force on them (`please_Compute_Porous_Slip_Velocity`). `.poroelastic` (`poroelastic`): `i alpha`, the Brinkman-type solid of Reloaded.
- `.tracer` (`tracers`): a count line, then `x y` per passive tracer; they move with the fluid and are written as `tracer.NNNN.vtk`.

THE OTHER FILES, BRIEFLY (2/2)

- `.concentration` (`concentration`): the initial scalar field as an $N_y \times N_x$ array of numbers (the reader builds a format of N_x floats per line); `.brinkman` (`brinkman`): the permeability field in the same layout.
- `.user_force` (`user_force_model`): one header line, then whatever your `give_Me_General_User_Defined_Force_Densities` expects (MATLAB reads it with `importdata`).
- `.coagulation` (`coagulation`): one number per line: the first index of the coagulating cells, a threshold radius, a threshold force, then the number of points in each cell.
- `.geo_connect`: `i j` pairs, without a count line, read whenever the file exists; the driver passes them to the force routines (the coagulation model and the concentration source use them). The output's connection file follows the springs, not this file.

Chapter 4e shows the muscle, porous and tracer examples; chapter 4c the update functions.

IB2d and IBAMR files, the short version (1/2)

Chapter 6 has the full comparison table; the points that matter when you move a structure between the two:

- Index base: IB2d/MATLAB 1, IB2d/Python 0, IBAMR 0.
- Several structures: one list in IB2d; one set of files per structure in IBAMR (`structure_names` in `input2d`).
- Springs: IB2d `i j k L [alpha]`; IBAMR `i j kappa L [fcn_idx [params]]`, the force function chosen by an index, not an exponent.
- Beams: IB2d's `.beam` is a torsional spring with a scalar `C`; IBAMR's `.beam` is IB2d's `.nonInv_beam` with `Cx Cy` (chapter 10).

IB2d and IBAMR files, the short version (2/2)

- Target points: IB2d `i k_t`; IBAMR `i kappa [eta]` with an optional damping; mass: IB2d `i k_m m`, IBAMR `i m kappa`.
- IBAMR only: `.xspring`, `.anchor`, `.rod`, `.director`, `.inst`, `.source`; IB2d only: `.d_spring`, `.muscle`, `.porous`, `.poroelastic`, `.tracer`, `.concentration`, `.user_force`, `.coagulation`, `.geo_connect`.
- Domain: IB2d's box starts at the origin and is periodic; IBAMR's goes anywhere, with wall and inflow conditions (chapter 7).
- Units: IB2d's examples are dimensionless in practice; IBAMR's examples mix. Chapter 8b.

Lecture 2's rubber-band generator writes both kinds from one script (a flag for the index base); MeshmerizeMe writes the `.vertex` file from a drawing, and the other files "must be manually created" (its paper; chapter 4f).

THE GENERATOR SCRIPT: RUBBERBAND.M / RUBBERBAND.PY

```
% Grid Parameters (MAKE SURE MATCHES IN input2d
    !!!)
Nx = 32;  Ny = 32;  Lx = 1.0;  Ly = 1.0;
N = 2*Nx;        % Number of Lagrangian Pts. (2x
    resolution)
a = 0.4;  b = 0.2;        % semi-axes
ds_Rest = 0;              % Resting length of
    springs
struct_name = 'rubberband';
[xLag,yLag] = give_Me_Immsersed_Boundary_Geometry
    (N,a,b);
plot(xLag,yLag,'r-'); hold on; plot(xLag,yLag
    ,'*');
print_Lagrangian_Vertices(xLag,yLag,struct_name);
k_Spring = 2.5e4;
print_Lagrangian_Springs(xLag,yLag,k_Spring,
    ds_Rest,struct_name);
```

```
function print_Lagrangian_Springs(xLag,yLag,
    k_Spring,ds_Rest,struct_name)
    N = length(xLag);
    spring_fid = fopen([struct_name '.spring'],
      'w');
    fprintf(spring_fid, '%d\n', N );
    for s = 1:N
        if s < N
            fprintf(spring_fid, '%d %d %1.16e
    %1.16e\n', s, s+1, k_Spring, ds_Rest);
        else
            fprintf(spring_fid, '%d %d %1.16e
    %1.16e\n', s, 1,   k_Spring, ds_Rest);
        end
    end
    fclose(spring_fid);
```

The top of the MATLAB script (left) and its spring writer (right); the Python twin writes the same files with `range(N)`: `s, s+1` and the closing `N-1, 0`, the 0-based files of slide 35.

THE GENERATOR: HOW TO USE IT

- The script knows the grid only to choose N; it is not called by the run, and `input2d` does not read it. Change either and keep them consistent by hand (Lecture 2: "if you change the generate_mesh file, you must run it").
- `give_Me_Immsersed_Boundary_Geometry` (sic) puts N points at equal angles on the ellipse centred at (0.5, 0.5). Write your own geometry function in any language; the files are plain text (Lecture 2 names Python, Julia and Excel on two of its slides).
- The same pattern writes `.beam` (one line per interior point, `s-1 s s+1`), `.target` (`s k`) and the rest: the example generators are the templates (chapter 4c builds one from scratch).
- Write the numbers with full precision (`%1.16e`, as the generators do): a point list rounded to four decimals at $\Delta x = 1/512$ has visible kinks.
- Keep the plot the generator draws: it is the first check of the geometry, and chapter 4c's exercise starts from it.

CHECK THE GEOMETRY BEFORE YOU RUN

- `test_IB2d_IBAMR_Geometry.m` (Appendix A; `examples/support/matlab-exercises` in this repository, with its Python version `test_ib_geometry.py`): reads a `.vertex` and a `.spring` file and makes two figures: the points with their index labels, and the points with the spring connections (alternating colours, so a missing or doubled spring stands out). Two lines to set at the top: `flag_IBAMR = 0` for 1-based files (MATLAB IB2d; Lecture 2's `test_IB_Geometry_and_Springs`), 1 for Python IB2d or IBAMR files, which count from 0; and `numCols`, the number of columns of the `.spring` file. The Python version: `python3 test_ib_geometry.py NAME [--ibamr]` saves `NAME_points.png` and `NAME_springs.png`.
- `read_Vertex_Points_and_Plot_Them.m` (the IB2d repository, `data_analysis/analysis_in_matlab/DA_Blackbox`): the vertices only.
- Look for: a spring that is missing (the band not closed: a classic off-by-one, chapter 6), a point outside $[0, L_x] \times [0, L_y]$, points closer than $\Delta x/4$ or farther than Δx (compute Δs from the file; the checker does not), two structures that overlap.
- A second check costs nothing: run with `Tfinal` set to a few Δt and `print_dump = 1`, and open dump 0000 (chapter 4b). The initial frame shows the structure as the code read it, with the connections it will draw.

THE OUTPUT: viz_IB2d/ AND hier_IB2d_data/

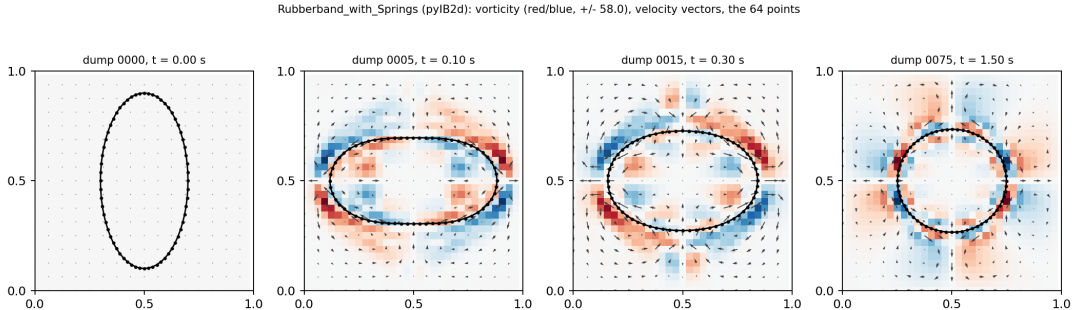
```
viz_IB2d/
  lagsPts.0000.vtk        # the points
  lagPtsConnect.0000.vtk  # the points joined
     along the springs (springs = 1)
  Omega.0000.vtk          # vorticity          (
     save_Vorticity)
  P.0000.vtk              # pressure           (
     save_Pressure)
  u.0000.vtk              # velocity vectors   (
     save_uVec)
  uMag.0000.vtk           # |u|                (
     save_uMag)
  uX, uY, fX, fY, fMag    # when their flags are
     1
  tracer.0000.vtk         # with tracers
  concentration.0000.vtk  # with a concentration
  ... one set per dump: 0001, 0002, ...
hier_IB2d_data/           # with save_hier = 1
  fX_Lag.0000.vtk  fY_Lag.0000.vtk  fMag.0000.vtk
  fNorm, fTan             # normal/tangential (
     MATLAB, > 5 points)
```

- Dump `0000` is $t = 0$; then one dump every `print_dump` steps, numbered consecutively: $t =$ dump $\times$ `print_dump` $\times \Delta t$.
- 76 dumps of the 32^2 example: 456 files, 6.7 MB in `viz_IB2d` (Omega 15 KB, u 33 KB, points 3 KB per dump) and 0.9 MB of forces. Text files: the size grows with $N_x N_y$.
- With damped springs instead of springs, Python writes no connection file and MATLAB writes `damp_Connect.NNNN.vtk`. MATLAB adds `viz_IB2d/Data_For_Restart/` for poroelastic runs.

THE OUTPUT FILES' FORMAT

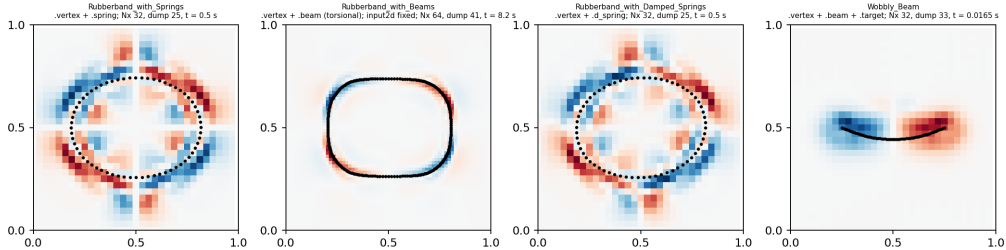
- All files are VTK legacy text. The Python writer with the `vtk` module (the runs here) writes "# vtk DataFile Version 5.1" with the `OFFSETS/CONNECTIVITY` cell layout; MATLAB's writers and Python's fallback writer (no `vtk` module) write "Version 2.0" with the classic `CELLS` block. Either way: the fields as `STRUCTURED_POINTS` with `DIMENSIONS Nx Ny 1` (except some velocity dumps, written N_y N_x: dump 0 of `u`, `uX`, `uY` with the `vtk` module, the later `u` dumps with the fallback writer; chapter 4f), `SPACING dx dy 1`, `ORIGIN 0 0 0` and `POINT_DATA` (`SCALARS Omega double`, `VECTORS u double`); the points as `UNSTRUCTURED_GRID` with vertex cells (`lagsPts`) or line cells (`lagPtsConnect`, following the springs); the forces as scalars on those points.
- The grid values are on the lattice $x_i = i\,\Delta x$, $i = 0 \ldots N_x - 1$ (no $x = L_x$ column: periodic).
- VisIt and ParaView open them directly (chapter 4b: group as `Omega.*.vtk`); chapter 33's `read_vtk_legacy.py/.m` reads them in Python and MATLAB/Octave (it reads both cell layouts); the repository's `data_analysis` folder has IB2d's own readers.
- The Python writer uses the `vtk` module when it is installed, its own text writer otherwise, with the same layout; the MATLAB writer is its own.

A FIRST LOOK AT THE RUN



- The band starts as an ellipse (semi-axes 0.4, 0.2) with zero-rest-length springs; it pulls itself towards a circle, overshoots, and rings down while the fluid it drives spins up four vortices in the corners of the periodic box. Peak speed 4.5 over the run; $\mathrm{Re} = \rho U a/\mu \approx 180$ at most.
- Read from the files with chapter 33's reader in a few lines; the pictures of this chapter are matplotlib on the `.vtk` files. Chapter 4b does the same in VisIt, with the time slider, and chapter 4d uses this run for the resolution study.

SAME POINTS, DIFFERENT FILES



- Springs, torsional beams, damped springs, and a beam with two target points: the four Python rubber-band examples a third of the way through their runs, vorticity and points. The structure file decides the physics. The springs and damped-springs runs share the 64-point ellipse; the beams example has its own 249-point ellipse with the axes swapped (0.4 along x) on a 64^2 grid; the wobbly beam is a line.
- The beams-only band (second panel) moves slowly: its constant rest value C does not match the ellipse (previous slide), so it relaxes towards a rounded square, with $|\mathbf{u}|$ up to 0.26 at $t = 0.4$, $|\omega|$ up to 10 at $t = 0.2$, the half-widths going from 0.40/0.20 to 0.32/0.25, and the vorticity below 1 by $t = 8$ s. That panel is from a corrected `input2d`: next slide.

A TRAP, FOUND WHILE MAKING THE PREVIOUS PICTURE

The shipped `pyIB2d/Examples/Rubberband_with_Beams/input2d` (2024 clone) has

```
beams = 0                % 0 (for no) or 1 (for yes)
update_beams = 0
nonInv_beams = 1         % <-- not a key the code knows
```

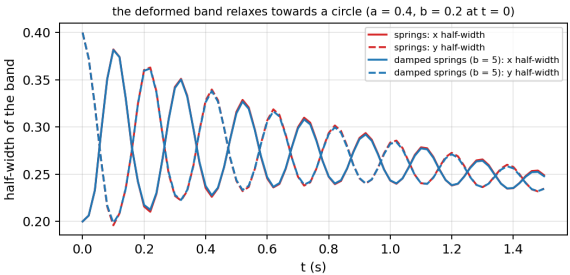
- The parser knows `nonInvariant_beams`, not `nonInv_beams`, and ignores unknown keys. Result: no fibre model at all, “FIBER MODEL INCLUDES:” followed by nothing, and 25 s of simulated time in which nothing moves (`max |u| = 0` in every dump, 8.6 s of wall time).
- The `.beam` file has five columns, so the intended flag is `beams = 1`; with that the band moves (slide 56).
- Worth a note to the IB2d repository: a one-line fix in the example, and a warning in the parser for unknown keys would catch the whole class of error.

WHAT TO TAKE FROM THE TRAP

- The console is the check: every switched-on model is listed under “FIBER MODEL INCLUDES”. A missing line means a missing or misspelt flag.
- A run that produces output is not a run that did what you meant. Open dump 0001 and look for motion.
- The flag names are the keys of `please_Initialize_Simulation` (`.m`/`.py`); copy them from a working example rather than typing them.
- A second copy of a key in the same block does not override the first in Python (the test of slide 27): edit the existing line.

> The same habit serves IBAMR: its input database also accepts keys it never reads and does not warn; the examples print the database into the log with the unused keys marked (chapter 6’s log slides). Print or log what the code read, and look at the first frame.

THE BAND RELAXES: A NUMBER FROM THE OUTPUT



- Half-widths of the point cloud in x and y from `lagsPts.NNNN.vtk`: 0.2 and 0.4 at $t = 0$, both near 0.24 by $t = 1.5$ (a circle of about the initial area, $\sqrt{0.2 \cdot 0.4} = 0.28$, less the leak through the 4-cell-thick boundary at this coarse resolution).
- With $b = 5$ the damped band's curve lies on the undamped one (differences below 0.002): at this stiffness and resolution the fluid damps the band faster than the dashpots do, and a larger b would be needed to see them.
- This is the kind of quantity to track in a study: one number per dump, from the points file, plotted against time and compared across grids (chapter 4d) or stiffnesses. Chapter 33's reader and IB2d's `data_analysis` scripts both do the file reading.

UNITS AND SCALE

- The comments in `input2d` say N s/m^2, kg/m^3 (one example file says kg/m^2) and metres, but the examples are dimensionless in effect: $\mu = 0.01$, $\rho = 1$, $L = 1$, a band of size 0.4 and stiffness $2.5 \cdot 10^4$ reproduce no particular animal. Treat the numbers as a Reynolds number and a dimensionless stiffness (chapter 8b).
- For the rubber band: peak speed about 4.5, so $\mathrm{Re} = \rho U a/\mu \approx 1 \cdot 4.5 \cdot 0.4/0.01 \approx 180$ at the peak and far lower during most of the run; a viscous, laminar wobble.
- To model a real organism, either scale its physics to these units (length by L, time by L/U, so that μ becomes $1/\mathrm{Re}$ and the stiffnesses become dimensionless groups) or enter SI values throughout and check that Δt still works. Chapter 8a shows what changing one parameter does and what it costs.
- The stiffnesses in the files are per point (force per unit stretch, applied at a point); doubling the number of points on the same structure halves the force each carries. Lecture 9's `kappa_spring*ds/(ds^2)` is the way to keep the elastic property of the structure fixed under refinement; chapter 6 derives it.

WHAT GOES WRONG FIRST (ALL SEEN IN THIS CHAPTER'S RUNS OR READ IN THE CODE)

- An unknown or misspelt flag is ignored: no model, no warning (slide 57). A duplicate key: first wins (Python) or "Improperly Declared" (MATLAB).
- Indices off by one between MATLAB and Python files, or from a loop that starts at the wrong number (slide 35); a band that is not closed.
- `string_name` not matching the files: MATLAB's `fopen` returns -1 and `textscan` fails with an invalid file identifier; Python raises `FileNotFoundError`. Check the quotes and the folder you run in.
- A row with too many values: MATLAB's `textscan` reads the whole file as one stream and shifts everything after it (a missing last value becomes `NaN` and is harmless); Python's `genfromtxt` stops with its message ("Check that all rows . . . have the same number of columns").
- The generator was changed but not run: the run uses the old files.

WHAT GOES WRONG FIRST (2)

- Odd `Nx` or `Ny`: the FFT operators are built for even counts (“MUST BE EVEN”).
- Δt or a stiffness too large: “BLOW UP! (*forces TOO large*) -> try decreasing the time-step or decreasing material property stiffnesses” from `please_Move_Lagrangian_Point_Positions`, often at the first step (chapter 4d).
- Points crossing the edge re-enter from the other side and may blow up (slide 28).
- Python without numba runs, slowly; MATLAB R2020a/b cannot draw the live figure (set `plot_Matlab = 0` and use VisIt); Octave is not supported (slide 15).
- The output folders are reused: a second run in the same folder overwrites `viz_IB2d/` dump by dump; a shorter second run leaves old dumps behind at the end. Delete the folders or run in a copy.

EXERCISE: YOUR OWN RUBBER BAND (HOMEWORK 4 OF THE UA COURSE)

1. Run `Rubberband_with_Springs` in MATLAB or Python as shipped. Note the wall time and the number of dumps. Open `lagsPts.0000.vtk` in a text editor and find the first point of the `.vertex` file in it.
2. Copy the folder. In the generator, make the band 50 % smaller and deform it differently (stretched in y instead of x, or a square, or a circle with a dent); keep N so that $\Delta s \approx \Delta x/2$ along the new shape. Run the generator, run the checker of slide 52, then run `main2d`.
3. Plot the structure at $t = 0$ and at the first dump after it, with the velocity vectors and the vorticity (matplotlib or MATLAB now, VisIt after chapter 4b), and write two sentences on what the band did.
4. Change one flag: turn the springs into damped springs (write the `.d_spring` file from the `.spring` one, add a column, switch the flags) and compare the half-width curve of slide 59.
5. If you use the Python version, convert the MATLAB `NL_Rubberband_With_Springs` example (nonlinear springs, $\alpha = 2$) to it: the files need the index shift of slide 35, `input2d` needs nothing.

The 2021 variant of the homework (Lecture 11): run an example related to your project instead, and hand in the same two pictures. Appendix C collects the homework bank.

SUMMARY

- IB2d is a complete 2D immersed boundary code in MATLAB and Python: one periodic grid solved by FFT, one list of points carrying springs, beams, target points, masses, muscles, porosity and more, with text input and VTK output; the right tool for learning the method and for elastic 2D models at moderate Reynolds number, on a laptop.
- An example is a folder: `main2d`, `input2d`, a generator script, and the structure files named by `string_name`; the run writes `viz_IB2d/` and `hier_IB2d_data/`.
- `input2d` is six braced blocks of `key = value`; keys are looked up by name, unknown keys are ignored, missing flags are off. The console's "FIBER MODEL INCLUDES" is the check that the flags were read.
- The structure files share one layout: a count line, then one entry per line; `.vertex` is required, the rest are switched on by their flags. The MATLAB files count from 1, the Python files from 0, IBAMR's from 0.
- The force laws are simple and are on the slides: power-law springs, damped springs, torsional beams with a scalar rest cross product, non-invariant beams with a rest curvature vector, tethered target points, massive shadow particles.
- Next: chapter 4b runs the examples and opens the output in VisIt; 4c moves boundaries with the update functions and builds a geometry from scratch; 4d is the stability and convergence lab; 4e the gallery and the muscle models; 4f MeshmerizeMe.

REFERENCES AND CREDITS

- N. A. Battista, A. J. Baird and L. A. Miller, *Integr. Comp. Biol.* 55:901–911 (2015); N. A. Battista, W. C. Strickland and L. A. Miller, *Bioinspir. Biomim.* 12:036003 (2017), arXiv:1610.07944; N. A. Battista, W. C. Strickland, A. Barrett and L. A. Miller, *Math. Meth. Appl. Sci.* 41:8455–8480 (2018), arXiv:1707.06928; M. Santiago, N. A. Battista, L. A. Miller and S. Khatri, *Bioinspir. Biomim.* 17:036003 (2022); D. M. Senter et al., *Bioinspir. Biomim.* 16:016008 (2021).
- C. S. Peskin, The immersed boundary method, *Acta Numerica* 11:479–517 (2002): the scheme and the 4-point delta function that IB2d implements.
- The IB2d repository, https://github.com/nickabattista/IB2d (GPL-3.0): `README.md`, `IB2d_New_Functionality.txt`, `pyIB2d/README.txt`, `matIB2d/Examples/EXAMPLE_LIST.txt`; the readers `IBM_Driver.m/.py` (functions `read_*_Points`), `please_Read_input2d_File`, `please_Initialize_Simulation`; the force laws in `please_Find_Lagrangian_Forces_On_Eulerian_grid.m`; the kernel in `give_Delta_Kernel.m`; the fluid solver's header in `please_Update_Fluid_Velocity.m`. Code as of the 2024-03-06 clone; the GitHub page as of October 2026.
- Runs: the four Python rubber-band examples in WSL (Ubuntu 24.04, Python 3.12, numpy 1.26, numba 0.68, no pyfftw), `wsl/final/ib2d_*.sh` and `ch4a_*.sh`; figures by `tools/ch4a_figures.py` with `examples/analysis/read_vtk_legacy.py`. The Octave attempt: `ib2d_setup2.sh`, `ib2d_setup3.sh`.
- Sources merged (slides 4–5): "Getting Running on IB2D and IBAMR" (2016); UA MATH 481/581 Lectures 9, 10 and 11 (2021–24); "Introduction to IB2D" (2024); "IB2d and IBAMR associated files" (Lecture 2, 2022). The boundary description and the input-file paragraphs on slides 9, 24 and 42 keep their wording from those decks and from the IB2d documentation they quote.